



Chiusure

Funzioni lambda

Lezione 13+
Corso 1



Programmazione Funzionale

- Paradigma di programmazione che descrive che cosa deve essere fatto invece di specificare come farlo. Si contrappone alla programmazione imperativa.
- Tratta il calcolo come la valutazione di funzioni matematiche
- Le caratteristiche principali della programmazione funzionale includono:
 - Funzioni pure: le funzioni non hanno effetti collaterali e producono sempre lo stesso output con gli stessi input
 - Immutabilità: i dati non possono essere modificati dopo la creazione, ma si creano nuove strutture dati
 - Funzioni di ordine superiore: le funzioni possono ricevere altre funzioni come argomenti e restituire funzioni come risultato
 - Ricorsione: utilizzata al posto dei cicli iterativi
 - Valutazione pigra: le espressioni vengono valutate solo quando necessario.
- Rust non è un linguaggio di programmazione puramente funzionale (come Haskell), ma offre molti concetti e tecniche della programmazione funzionale:
 - Funzioni possono essere trattate come variabili
 - Immutabilità dei dati
 - Funzioni di ordine superiore
 - Espressioni e non istruzioni
 - Pattern matching
 - Tipi di dati algebrici.
- La programmazione funzionale in Rust si manifesta attraverso:
 - l'uso intensivo degli iteratori
 - Closures
 - Option e Result
 - Immutabilità.

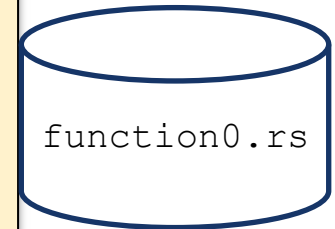
Funzioni come variabili

- Molti linguaggi offrono la possibilità di trattare una funzione come un tipo di dati qualsiasi, consentendo di memorizzarla in una variabile
 - Nel linguaggio C questo è possibile modellando la variabile come un puntatore a funzione, usando la sintassi
TipoRitornato (* v) (p1: Tipo₁, ..., pn: Tipo_n);
che definisce **v** come puntatore ad una funzione che accetta n parametri e restituisce un valore dei tipi indicati
 - In C++, è possibile anche modellare la variabile come oggetto funzionale (ovvero che definisce il metodo ***operator() (...)*** e che può avere uno stato)
 - In Rust è possibile assegnare ad una variabile il puntatore ad una funzione (che avrà come tipo ***fn(T₁, ..., T_n) -> U***) o assegnare un valore che implementa un tratto funzionale: ***FnOnce***, ***FnMut***, ***Fn***

Puntatori a funzione

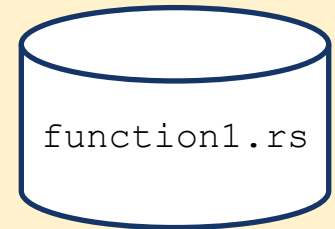
- Qualunque sia la natura del dato assegnato, si utilizza la variabile che contiene il puntatore come se fosse essa stessa una funzione
 - Sia il tipo di ritorno che tutti i tipi degli argomenti devono corrispondere a quanto dichiarato nella definizione della variabile (e della funzione)

```
fn f1(i: i32, d: f64) -> f64 {  
    return i as f64 * d  
}  
fn main() {  
    let ptr = f1; // variabile puntatore a funzione  
  
    let num = ptr(2, 3.14);  
  
    println!("{}", num);  
}
```



```
fn add(a: i32, b: i32) -> i32 { a + b }  
fn subtract(a: i32, b: i32) -> i32 { a - b }  
fn multiply(a: i32, b: i32) -> i32 { a * b }
```

```
fn main() {  
    let (x, y) = (10, 5);  
  
    // Selezioniamo la funzione in base a una condizione.  
    let operation_name = "add";  
  
    // Assegnamo il puntatore alla funzione corrispondente.  
    let operation: fn(i32, i32) -> i32 = match operation_name {  
        "add" => add,  
        "subtract" => subtract,  
        "multiply" => multiply,  
        _ => panic!("Operazione non supportata"),  
    };  
  
    let result = operation(x, y);  
  
    println!("Il risultato dell'operazione '{}' è: {}", operation_name, result);  
}
```



```
fn add_one(x: i32) -> i32 {  
    x + 1  
}  
  
fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {  
    f(arg) + f(arg)  
}  
  
fn main() {  
    let answer = do_twice(add_one, 5);  
    println!("The answer is: {}", answer);  
}
```



Funzioni lambda

- Una funzione lambda è una funzione anonima costituita da un blocco di codice espresso in forma letterale
 - Tale forma dipende dal linguaggio di programmazione ed è oggetto delle forme più disparate
- In **Rust** si racchiudono i parametri formali tra `|` `|`:
 - `let f = | v | { v + 1 }`

Funzioni lambda

- Una **funzione lambda**, una volta **definita** ed **assegnata** ad una **variabile**, può essere invocata trattando la variabile come se fosse una funzione
 - Ovvero facendo seguire, al nome della variabile la lista degli argomenti racchiusi in parentesi tonde: ad esempio, **f(5)**;

```
fn main() {  
    let answer = | v | { v + 1 };  
  
    println!("The answer is: {}", answer(5));  
}
```



Chiusure

- Il corpo di una funzione lambda può fare riferimento alle variabili che sono visibili nel contesto in cui è definita, acquisendone un riferimento, una copia o il possesso completo, in base al linguaggio e alla sintassi usata
 - Tali variabili, che compaiono nel corpo della funzione lambda sono dette **variabili libere**
- La funzione lambda così ottenuta viene detta **chiusura**
 - In quanto racchiude, al proprio interno, (una copia de) i valori catturati (quelli contenuti nelle variabili libere), rendendoli disponibili quando sarà successivamente invocata

Terminologia



- **Funzioni di ordine superiore:** funzioni i cui parametri e/o il cui tipo di ritorno sono a loro volta una funzione
- **Funzione anonima:** funzione definita senza un nome e il loro uso è diretto (ossia la dichiarazione viene scritta dentro il codice)
 - Sintassi più concisa: il compilatore è in grado di inferire (ossia di riconoscere) i tipi degli argomenti e del valore di ritorno
- **Funzione lambda:** sinonimo di funzione anonima
- **Chiusura:** Funzione lambda (ossia anonima) che può catturare variabili (detti **variabili libere**) dell'ambiente circostante

Cattura delle variabili libere

- Per default in Rust, tutte le variabili libere che compaiono nel corpo di una funzione lambda sono **catturate per riferimento**
 - Il compilatore, automaticamente, crea un prestito in lettura (**&**)
 - Se occorre modificare il contenuto delle variabili catturate (acquisendole con **&mut**), occorre dichiarare la funzione lambda come mutabile (**let mut f = |...| {...};**)
 - Il borrow checker verifica che tali riferimenti siano coerenti tra loro e con il tempo di vita dei valori cui si riferiscono
- Se occorre, è possibile indicare che la funzione lambda deve acquisire il **possesso dei valori** contenuti nelle variabili libere
 - Lo si fa antepoendo alla definizione della funzione lambda la parola chiave **move**
 - **let f = move |...| {...};**
 - Il compilatore può inferire che la variabile libera viene mossa e consumata.

Esempi di Chiusure: quali sono le variabili libere?

```
fn main() {  
    let add = |num1, num2| num1 + num2;  
    println!("La somma è: {}", add(1, 2));  
}
```

closure1.rs

```
fn main() {  
    let factor = 2;  
    let multiply = |n| n * factor;  
    println!("Il risultato è: {}", multiply(5));  
}
```

closure2.rs

```
fn main() {  
    let multiply = |x: i32, y: i32| -> i64 {  
        (x * y).into() };  
    println!("{}", multiply(3, 4)); // prints 12  
}
```

closure3.rs

```
fn main() {  
    let add = |num1, num2| num1 + num2;  
    println!("La somma è: {}", add(1, 2));  
}
```

closure1.rs

```
fn main() {  
    let factor = 2;  
    let multiply = |n| n * factor;  
    println!("Il risultato è: {}", multiply(5));  
}
```

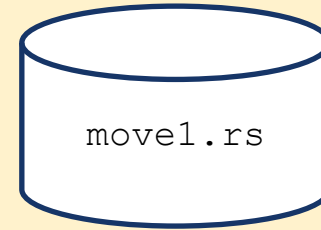
Variabile libera
catturata

closure2.rs

```
fn main() {  
    let multiply = |x: i32, y: i32| -> i64 {  
        (x * y).into() };  
    println!("{}", multiply(3, 4)); // prints 12  
}
```

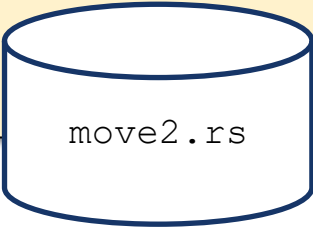
closure3.rs

```
fn main() {  
    let mut count = 0;  
  
    let mut increment = move || {  
        count += 1; // Incrementiamo la variabile catturata  
        println!("Il conteggio è: {}", count);  
    };  
  
    increment();  
  
    increment();  
}
```



- Si noti che questa closure non rappresenta una funzione pura perché ogni volta che viene chiamata (con lo stesso parametro di input) genera sempre valori diversi.

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
  
    let print_numbers = move || {  
        println!("I numeri sono: {:?}", numbers);  
        // La chiusura usa il vettore catturato  
        // numbers è stato mosso (moved) dentro la chiusura  
    };  
  
    // non può più essere usato qui  
    //println!("I numeri sono: {:?}", numbers);  
    // Questo darebbe errore di compilazione  
  
    print_numbers(); // Chiamiamo la chiusura  
}
```



move2.rs

```
fn main() {
```

```
    let mut count = 0;
```

mut necessario perché viene
modificata la variabile libera
posseduta

```
    let mut increment = move || {  
        count += 1; // Incrementiamo la variabile catturata  
        println!("Il conteggio è: {}", count);  
    };
```

```
    increment();
```

```
    increment();  
    println!("Hello, {}", count);
```

```
}
```



```
Il conteggio è: 1  
Il conteggio è: 2  
Hello, 0
```

Come mai funziona così?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>


```
fn main() {
```

```
    let mut myBox = Box::new(42);
```

```
    let mut increment = move || {  
        *myBox += 1; // Incrementiamo la variabile catturata  
        println!("Il conteggio è: {}", *myBox);  
    };
```

```
    increment();
```

```
    increment();  
    println!("Hello, {}", *myBox);
```

```
}
```

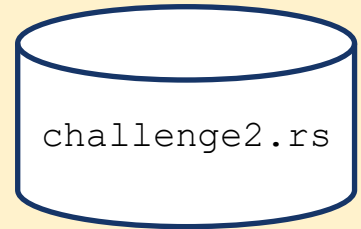


Questo programma non compila? Come mai?
Che cosa cambia rispetto al programma precedente?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

```
fn main() {  
    let mut count = 0;  
  
    let mut increment_n = |n| {  
        count += n; // Incrementiamo la variabile catturata  
        println!("Il conteggio è: {}", count);  
    };  
  
    increment_n(10); // Stampa: Il conteggio è: 10  
  
    increment_n(5); // Stampa: Il conteggio è: 15  
  
    println!("Hello, {}", count);  
}
```



```
Il conteggio è: 10  
Il conteggio è: 15  
Hello, 15
```

Come mai funziona così?

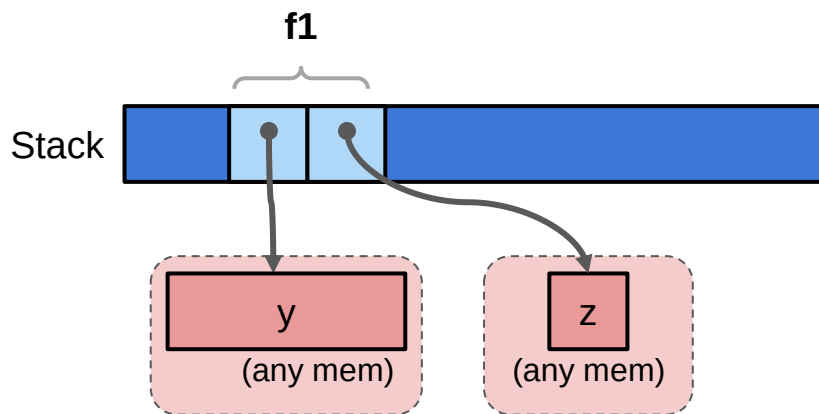
Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

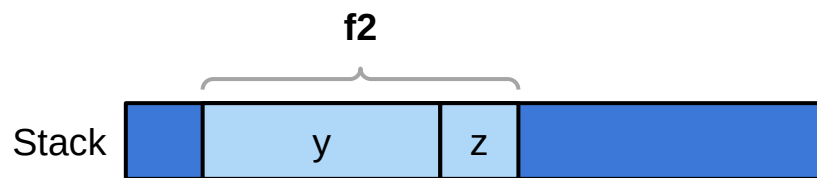
Memorizzazione della chiusura

- In Rust, il compilatore trasforma una chiusura in una tuple struct, avente tanti campi quante sono le variabili libere
 - Le variabili libere possedute (con move) vengono mosse all'interno della struct
 - Le variabili libere non possedute copiano il riferimento all'interno della struct.

```
let f1 = | x | { x + y + z };
```



```
let f2 = move | x | { x + y + z };
```



I tratti funzionali

- Rust definisce tre tratti funzionali che possono essere implementati **solo** tramite chiusure
 - Quale tratto venga implementato, dipende da **cosa** e **come** viene catturato

```
trait FnOnce<Args> {  
    type Output;  
    fn call_once(self, args: Args) -> Self::Output;  
}  
  
trait FnMut<Args>: FnOnce<Args> {  
    fn call_mut(&mut self, args: Args) -> Self::Output;  
}  
  
trait Fn<Args>: FnMut<Args> {  
    fn call(&self, args: Args) -> Self::Output;  
}
```

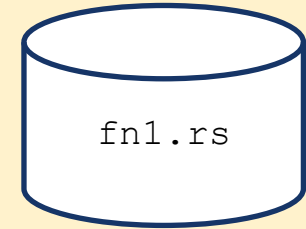
Come Rust determina quale tratto implementare

- Il compilatore di Rust inferisce automaticamente quale tratto o quali tratti funzionali una closure implementa analizzando cosa fa la closure con le variabili catturate.
 - Se la closure muove una variabile catturata e la consuma, implementa solo **FnOnce**
 - Se la closure muta una variabile catturata per riferimento mutabile, implementa **FnMut** e **FnOnce**
 - Se la closure prende in prestito le variabili (per riferimento immutabile) o non cattura nulla, implementa **Fn**, **FnMut** e **FnOnce**.

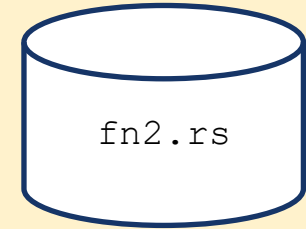
Tratti funzionali

- Una chiusura che implementa il tratto **Fn<Args>** si limita ad accedere in sola lettura alle variabili libere (&)
 - Finché la funzione esiste, le variabili libere sono in prestito condiviso e non possono essere cambiate
 - Pertanto, la chiusura può essere invocata un numero qualsiasi di volte e produce, a parità di argomenti, sempre lo stesso risultato
- Una chiusura che implementa il tratto **FnMut<Args>** può essere invocata più volte, ma ha catturato una o più variabili in modo esclusivo (&mut)
 - Questo tipo di chiusura produce effetti collaterali ed ha pertanto uno **stato**
 - Esecuzioni successive con gli stessi parametri in ingresso possono dare risultati differenti
- Una chiusura implementa il tratto **FnOnce<Args>** se consuma uno o più valori come parte della propria esecuzione
 - Pertanto, potrà essere invocata una sola volta.

```
fn main() {  
    let greeting = "Ciao";  
  
    // Definiamo una closure che non modifica variabili esterne  
    let greet = || {  
        println!("{}", greeting);  
    };  
  
    // Chiamiamo la closure e otteniamo sempre lo stesso risultato  
    greet();  
    greet();  
    greet();  
}
```



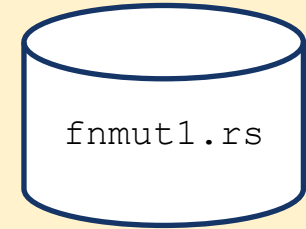
```
fn main() {  
  
    let x = 10;  
    // Definiamo una closure che non modifica variabili esterne  
    // implementa il tratto Fn  
  
    let aggiungi_a_x = |y: i32| {  
        println!("Il risultato di x + y è: {}", x + y);  
    };  
  
    aggiungi_a_x(5); // Chiamiamo la chiusura con il parametro 5  
    aggiungi_a_x(10); // Chiamiamo la chiusura con il parametro 10  
    aggiungi_a_x(10); // Chiamiamo la chiusura con il parametro 10  
}
```



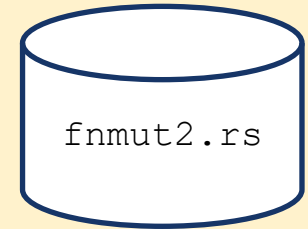
```
Il risultato di x + y è: 15  
Il risultato di x + y è: 20  
Il risultato di x + y è: 20
```

a parità di argomento,
si ottiene lo stesso
risultato


```
fn main() {  
    let mut counter = 0;  
  
    // Definiamo una closure che può essere chiamata più volte  
    let mut increment_counter = || {  
        counter += 1;  
        println!("Il contatore è ora: {}", counter);  
    };  
  
    // Chiamiamo la closure più volte  
    increment_counter();  
    increment_counter();  
    increment_counter();  
}
```



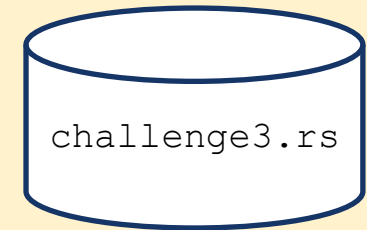
```
fn main() {  
    let mut x = 10;  
  
    // Una chiusura che implementa il tratto FnMut e accetta un parametro  
    let mut aggiungi_a_x = |y: i32| {  
        x += y;  
        println!("Il nuovo valore di x è: {}", x);  
    };  
  
    aggiungi_a_x(5); // Chiamiamo la chiusura con il parametro 5  
    aggiungi_a_x(7); // Chiamiamo la chiusura con il parametro 7  
}
```



```
fn main() {  
    let vec = vec![1, 2, 3, 4, 5];  
  
    // Definiamo una chiusura che prende il possesso del vettore vec  
    let consume_vector = || {  
        // Consumiamo il vettore stampando tutti i suoi elementi  
        for num in vec {  
            println!("{}", num);  
        }  
    };  
  
    // Chiamiamo la chiusura. Dopo questa chiamata, non possiamo più usarla.  
    consume_vector();  
  
    // Tentare di chiamare nuovamente la chiusura genererebbe un errore di compilazione  
    // consume_vector();  
}
```



```
fn main() {  
    let data = vec![1, 2, 3, 4, 5];  
  
    let process_data = move || { // move può anche essere omesso  
        let sum: i32 = data.iter().sum();  
        println!("La somma dei dati è: {}", sum);  
    };  
  
    // Chiamiamo la closure  
    process_data();  
  
    process_data();  
}
```

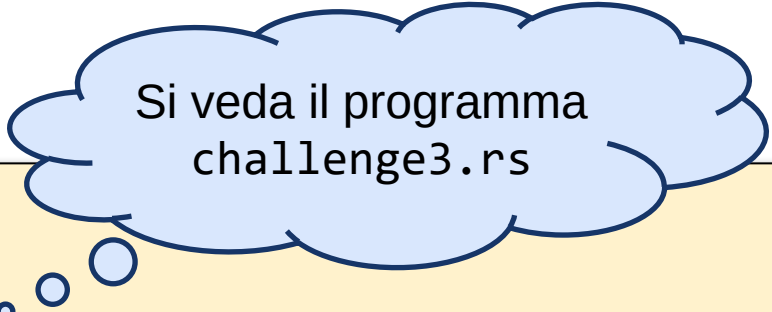


La somma dei dati è: 15
La somma dei dati è: 15

Come mai funziona così?

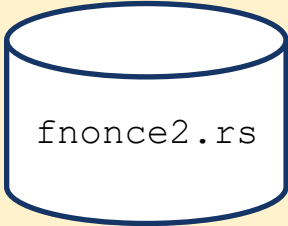
Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>



Si veda il programma
challenge3.rs

```
fn main() {  
    let data = vec![1, 2, 3, 4, 5];  
  
    // Definiamo una closure che può essere chiamata solo una volta  
    let process_data = move || {  
        let sum: i32 = data.iter().sum();  
        println!("La somma dei dati è: {}", sum);  
        drop(data);  
        // Consumiamo i dati per evitare ulteriori elaborazioni  
    };  
  
    // Chiamiamo la closure  
    process_data();  
  
    // Tentiamo di chiamare nuovamente la closure  
    // Questo produrrà un errore di compilazione perché la closure è stata consumata  
    //process_data();  
}
```



fnonce2.rs

```
fn main() {  
    let mut data = vec![1, 2, 3, 4, 5];  
  
    let mut process_data = move || {  
        data.push(6);  
        let sum: i32 = data.iter().sum();  
        println!("La somma dei dati è: {}", sum);  
    };  
  
    // Chiamiamo la closure  
    process_data();  
    data.push(7);  
}
```

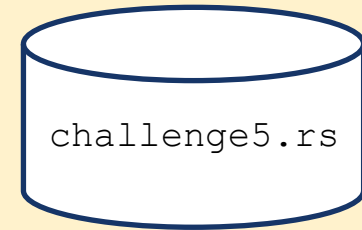


Funziona?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

```
fn main() {
    let range = 1..10;
    let f = || range.count();
    let n1 = f();
    println!("{}", n1);
    let n2 = f();
}
```



note: closure cannot be invoked more than once because it moves the variable `range` out of its environment

```
--> src/main.rs:3:16
```

```
3 |     let f = || range.count();
  |               ^^^^^
```

note: this value implements `FnOnce`, which causes it to be moved when called

```
--> src/main.rs:4:14
```

```
4 |     let n1 = f();
  |               ^
```

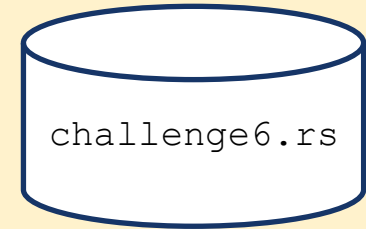
Il compilatore non compila come mostrato.

Come mai?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

```
fn main() {  
    let s = String::from("ciao");  
  
    // La closure prende possesso della variabile s  
    let my_closure = move || {  
        println!("Stampo la stringa: {}", s);  
        s  
    };  
  
    // my_closure implementa solo FnOnce  
    my_closure();  
    // Dopo questo punto, s non è più accessibile  
}
```



Perché `my_closure()` implementa solo `FnOnce`?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

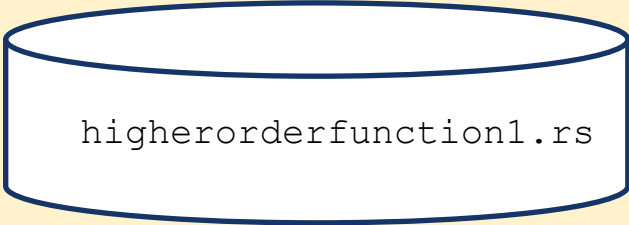
Funzioni di ordine superiore

- E' possibile implementare una funzione che accetta come parametro una funzione lambda ricorrendo alla programmazione generica
 - Ogni funzione lambda forma infatti un tipo a sé, la cui unica istanza è la funzione lambda stessa
 - Tuttavia, esse implementano tutte almeno uno dei tratti funzionali: è quindi possibile definire una funzione generica che accetta come parametro di ingresso un'istanza del tipo F, soggetta al vincolo che tale tipo deve implementare uno dei tratti funzionali
- Quale specifico tratto richiedere dipende dalla natura del codice che si intende scrivere:
 - FnOnce(Args) accetterà qualsiasi chiusura, ma questa potrà essere invocata una sola volta
 - FnMut(Args) e Fn(Args) sono via via più restrittivi sulle operazioni che è lecito eseguire all'interno della funzione λ

```
fn higher_order_function<F, T, U>(f: F) where F: Fn(T) -> U {  
    // ... codice che usa f(...)
}
```

```
fn multiply<F>(num: i32, func: F) -> i32
where
    F: Fn(i32) -> i32,
{
    func(num)
}

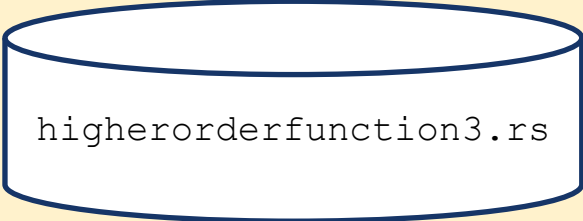
fn main() {
    let per_due = |x| x * 2;
    let per_tre = |x| x * 3;
    println!("10 x 2 = {}", multiply(10, per_due));
    println!("20 x 3 = {}", multiply(20, per_tre));
}
```



higherorderfunction1.rs

```
fn call_twice<F>(mut closure: F)
  where
    F: FnMut(),
{
  closure();
  closure();
}

fn main() {
  let mut i = 0;
  call_twice(|| i += 1 );
  println!("{}", i);
}
```



higherorderfunction3.rs

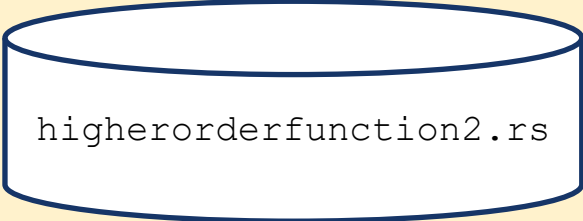
```
fn consume_closure<F>(f: F)
  where
    F: FnOnce() -> String,
{
  println!("La closure dice: {}", f());
}

fn main() {
  let text = "Hello, world!".to_string();

  let printer = || text;

  consume_closure(printer);

  // La seguente linea sarebbe errore se decommentata,
  // perché printer non può essere usata più di una volta.
  // consume_closure(printer);
}
```



higherorderfunction2.rs

Funzioni fn come parametro di Funzioni di Ordine Superiore

```
fn applica_funzione<F>(valore: i32, funz: F) -> i32
where
    F: Fn(i32) -> i32
{
    funz(valore)
}
fn raddoppia(x: i32) -> i32 {
    x * 2
}
fn main() {
    let triplica = |x| x * 3;

    println!("Raddoppiato: {}", applica_funzione(5, raddoppia)); // 10
    println!("Triplicato: {}", applica_funzione(5, triplica));   // 15
}
```

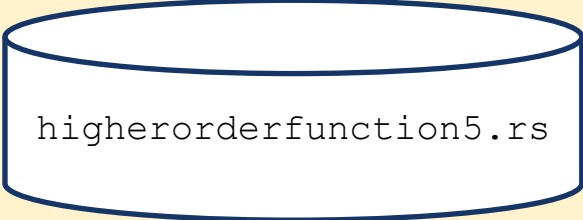


- Le funzioni normali (ossia definite con fn) implementano automaticamente tutti e 3 i tratti Fn, FnMut e FnOnce.

Factory di funzioni

- Analogamente, è possibile scrivere **una funzione che restituisce una chiusura**. Tale tipo di funzione di ordine superiore è detta factory di funzioni.
 - Se la chiusura ritornata cattura qualche variabile, occorre fare attenzione al fatto che, normalmente, ciò implica memorizzare un riferimento all'interno della chiusura stessa
 - Questo fatto potrebbe imporre restrizioni sul tempo di vita del dato catturato non facilmente compatibili con il fatto che la chiusura ritornata deve sopravvivere alla funzione stessa (e quindi a tutte le sue variabili locali e ai suoi argomenti)
 - In questa situazione, si usa comunemente il modificatore **move** per trasferire il possesso di tali dati alla chiusura stessa
 - Può, inoltre, essere necessario richiedere che il dato catturato sia clonabile, se l'esecuzione della chiusura ritornata tendesse a consumare il dato e non si volesse ridurla ad **FnOnce<Args>**

```
fn genera_contatore() -> impl Fn(i32) -> i32 {  
    let contatore = 0;  
    move |incremento: i32| contatore + incremento  
}  
  
fn main() {  
    let conto = genera_contatore();  
    println!("Il risultato è: {}", conto(3));  
    println!("Il risultato è: {}", conto(10));  
    println!("Il risultato è: {}", conto(10));  
}
```



higherorderfunction5.rs

```
Il risultato è: 3  
Il risultato è: 10  
Il risultato è: 10
```

```
fn generator(prefix: &str) -> impl FnMut() -> String {  
    let mut i = 0;  
    let b = prefix.to_string();  
    return move || {i+=1; format!("{}",b,i)}  
}  
  
fn main() {  
    let mut f = generator("id_");  
    for _ in 1..5 {  
        println!("{}",f());  
    }  
}
```

higherorderfunction4.rs

id_1
id_2
id_3
id_4


```
fn function_generator<T>(v: T) -> impl Fn() -> T
    where
        T: Clone,
{
    return move || { v.clone() };
}
```

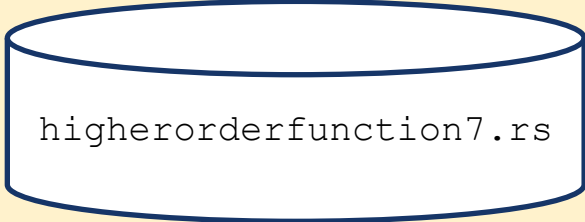
higherorderfunction6.rs

```
fn main() {
    // Crea una funzione che restituisce il numero 42
    let generate_42 = function_generator(42);
    // Crea una funzione che restituisce la stringa "hello"
    let generate_hello = function_generator("hello".to_string());

    // Esempio con un tipo più complesso come un vettore
    let vec = vec![1, 2, 3];
    let generate_vec = function_generator(vec);

    println!("Numero: {}", generate_42()); // Stampa "Numero: 42"
    println!("Numero: {}", generate_42()); // Stampa "Numero: 42"
    println!("Stringa: {}", generate_hello()); // Stampa "Stringa: hello"
    println!("Vettore: {:?}", generate_vec()); // Stampa "Vettore: [1, 2, 3]"
}
```

```
fn function_generator<T>(v: T) -> impl FnOnce() -> T
where
    T: Clone,
{
    return move || { v.clone() };
}
```



higherorderfunction7.rs

```
fn main() {
    // Crea una funzione che restituisce il numero 42
    let generate_42 = function_generator(42);
    // Crea una funzione che restituisce la stringa "hello"
    let generate_hello = function_generator("hello".to_string());

    // Esempio con un tipo più complesso come un vettore
    let vec = vec![1, 2, 3];
    let generate_vec = function_generator(vec);

    println!("Numero: {}", generate_42()); // Stampa "Numero: 42"
    println!("Stringa: {}", generate_hello()); // Stampa "Stringa: hello"
    println!("Vettore: {:?}", generate_vec()); // Stampa "Vettore: [1, 2, 3]"
    // Una seconda chiamata della funzione di chiusura ritornata causerebbe un
    // errore di compilazione perché la chiusura ha tratto funzionale FnOnce
}
```

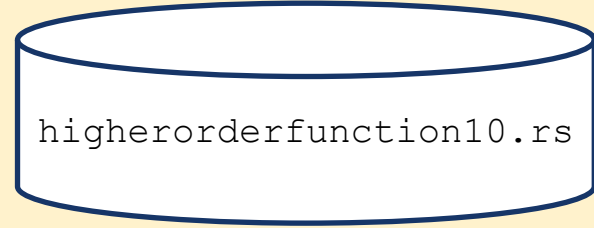
```
fn compose<F, G>(f: F, g: G) -> impl Fn(i32) -> i32
  where
```

```
    F: Fn(i32) -> i32,
```

```
    G: Fn(i32) -> i32,
```

```
{
  move |x| g(f(x))
}
```

```
fn main() {
  let add_then_double = compose(|n| n + 1, |n| n * 2);
  println!("{}", add_then_double(5));           // (5+1) * 2
  println!("{}", add_then_double(10));          // (10+1) * 2
}
```



12

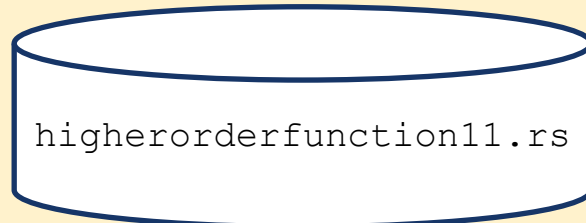
22

Restituire funzioni diverse

- Se si vogliono restituire tipi diversi di closure in base a una condizione, si usa un oggetto tratto.

```
fn crea_operazione(tipo: &str) -> Box<dyn Fn(i32) -> i32> {  
    match tipo {  
        "raddoppia" => Box::new(|x| x * 2),  
        "quadrato" => Box::new(|x| x * x),  
        _ => Box::new(|x| x),  
    }  
}
```

```
fn main() {  
    let op = crea_operazione("quadrato");  
    println!("{}", op(4)); // 16  
}
```



J4N



Per saperne di più

- Functional Programming using Rust
 - <https://medium.com/coderhack-com/functional-programming-using-rust-3776c10cfc6>
 - Introduzione alla programmazione funzionale in Rust con esempi
- Functional Programming In Rust: Unlocking Expressive Code
 - <https://marketsplash.com/tutorials/rust/rust-functional-programming/>
 - Rivisitazione degli aspetti funzionali presenti in Rust corredati da semplici esempi
- Easy Rust - Closures
 - https://dhghomon.github.io/easy_rust/Chapter_37.html
 - Presentazione del concetto di chiusura in Rust con approccio narrativo
- A guide to closures in Rust
 - <https://hashrust.com/blog/a-guide-to-closures-in-rust/>
 - Trattazione approfondita del concetto di chiusura in Rust con esempi e indicazioni di come il compilatore tratti tale tipo di dato

Glossario

- Programmazione funzionale: paradigma di programmazione che descrive che cosa deve essere fatto invece di specificare come farlo. Si contrappone alla programmazione imperativa. Tratta il calcolo come la valutazione di funzioni matematiche
- Funzione pura: Funzione che dato lo stesso input produce sempre lo stesso output e non ha effetti collaterali (ossia nessuna interazione con il mondo esterno come la modifica di variabili globali, l'output su schermo, la scrittura su file o il cambiamento degli argomenti passati)
- Funzione anonima: funzione che non ha un nome utilizzato per essere invocata
- Funzioni di ordine superiore: funzioni i cui parametri e/o il cui tipo di ritorno sono a loro volta una funzione
- Factory di funzioni: è una funzione di ordine superiore che restituisce una funzione
- Funzione lambda: sinonimo di funzione anonima
- Chiusura: Funzione lambda (ossia anonima) che può catturare variabili (dette variabili libere) dell'ambiente circostante
- Variabili libere: variabili dell'ambiente circostante utilizzate dalle chiusure